

MAHABOCW Medical Scholarship Verification Automation

Project Documentation / Context File (project.md)

Project Overview

This project automates the verification of medical scholarship claims submitted on the MAHABOCW portal.

The original workflow required a human verifier to:

1. Search a claim using its acknowledgement number.
2. Open the claim form.
3. Open each uploaded document one by one.
4. Read the Bonafide Certificate.
5. Determine the college name and address.
6. Compare it against the Excel sheet.
7. Update the corrected college details.

This process takes several minutes per application.

The goal of this project is to automate **everything except the manual login** while maintaining robustness against poor-quality applicant uploads.

Dataset

Input Excel:

Medical_Claim_Data_Part_1.xlsx

Worksheet:

Medical Data Verification-Par1

Important columns:

acknowledgement_no
corrected_college_name
corrected_college_address

Approximately:

21,000+ claims

Overall Architecture

The project has been redesigned into an Intelligent Document Processing (IDP) pipeline.

Excel
↓
Playwright
↓
Extract document URLs
↓
Download uploaded files
↓
Normalize documents
↓
OCR every page
↓
LLM determines page relevance
↓

LLM extracts college name & address

↓

Update Excel

The browser automation is intentionally isolated from document processing.

After download, Playwright is no longer involved.

Technology Stack

Browser Automation

- Playwright

HTML Parsing

- BeautifulSoup

Document Rendering

- PyMuPDF (fitz)

Image Processing

- Pillow

OCR

- PaddleOCR (GPU)

Language Model

- TBD (OpenAI / Gemini / Ollama / Local)

Data

- Pandas
-

Current Project Structure

```
project/
|
├─ verify_colleges.py
├─ document_processor.py
├─ classifier.py (temporary)
├─ ocr_engine.py (to build)
├─ extractor.py (to build)
├─ Medical_Claim_Data_Part_1.xlsx
|
└─ downloads/
    acknowledgement_number/
        bonafide.pdf
        college_id.pdf
        aadhaar.pdf
        ration.pdf
        self_declaration.pdf
        education_self_declaration.pdf
    pages/
        ...
```

Current Status

Stage 1

COMPLETE

Playwright logs in.

User manually:

- logs in
- opens Claims page
- opens filter

Automation:

- filters acknowledgement number
- opens claim form

Stable.

Stage 2

COMPLETE

Originally automation clicked every View Document button.

This was replaced.

Current implementation:

```
BeautifulSoup
```

```
↓
```

```
Read HTML
```

```
↓
```

```
Locate document labels
```

```
↓
```

Extract authenticated download URLs

No document viewer interaction anymore.

Only HTML parsing.

Much more reliable.

Supported Document Labels

The parser ONLY downloads the following six document fields.

Nothing else.

Bonafide certificate of current academic year

College Identity Card

Aadhaar card of child/spouse

Self Declaration

Ration Card

Education Self Declaration

This is intentional.

The page contains many hyperlinks.

They must NEVER be downloaded.

Only the six fields under

DOCUMENTS REQUIRED

are parsed.

Stage 3

COMPLETE

Downloads every uploaded document.

Supports

```
pdf
jpg
jpeg
png
```

Each claim stored as

```
downloads/
  acknowledgement/
    bonafide.pdf
    aadhaar.pdf
    ...
```

Missing uploads simply don't exist.

No exceptions thrown.

Duplicate Detection

Implemented.

Some applicants upload the exact same combined PDF into multiple fields.

Example

Bonafide

↓

combined.pdf

College ID

↓

combined.pdf

Aadhaar

↓

combined.pdf

SHA256 hash is calculated.

Duplicate files are skipped during normalization.

Only one copy is processed.

Stage 4

Mostly Complete.

document_processor.py

Responsibilities:

- detect file type
- render PDFs
- convert images
- split PDFs into pages

Current behaviour:

PDF

↓

page001.png

page002.png

page003.png

JPEG



PNG

Remaining Stage 4 Work

Current problem

Some mobile images are sideways.

Example

Portrait image

Actual document rotated 90°

Current renderer preserves this.

Need

Automatic orientation normalization.

Possible solutions

- OpenCV

or

- PaddleOCR angle classification

The recommendation is to rotate landscape pages before OCR.

Real-World Problems Discovered

During testing, multiple unexpected upload behaviours were found.

Problem 1

Applicants upload wrong document into wrong field.

Example

Bonafide uploaded into

Self Declaration

or

Ration Card

Solution

Ignore upload field.

Every uploaded page is OCR'd.

Problem 2

Applicants upload combined PDFs.

Example

Page 1
Declaration

Page 2
Bonafide

Page 3
Bonafide

Page 4
Student ID

Page 5
Aadhaar

Page 6
Ration

Sometimes

Same combined PDF uploaded into

Bonafide

College ID

Aadhaar

Self Declaration

Duplicate detection solves repeated processing.

OCR works page-by-page.

Problem 3

Missing uploads

Some applicants upload

only

Self Declaration

which is actually a 12-page combined PDF containing every document.

Example

Acknowledgement

2868131336

Current pipeline handles this correctly.

Only one PDF is processed.

Important Design Decision

The upload field is NOT trusted.

The upload field is merely metadata.

The actual page contents determine relevance.

Original Classification Plan

Originally

classifier.py

attempted to classify pages into

Bonafide

College ID

Aadhaar

Ration

Self Declaration

using keyword scoring.

This has now been abandoned.

Reason

We do NOT care what every page is.

We ONLY care whether the page contains

Bonafide

or

Student Identity Card

Everything else can be discarded.

Simplified Final Pipeline

Every page

↓

OCR

↓

LLM decides

↓

Relevant?

↓

YES

↓

Extract

↓

Write Excel

Otherwise

↓

Discard

No document classification stage is required.

Remaining Work

Step 1

Create

```
ocr_engine.py
```

Responsibilities

```
image
↓
PaddleOCR
↓
raw text
```

Simple wrapper.

No filtering.

No extraction.

No AI logic.

Only OCR.

Step 2

After every page is rendered

```
page001.png
```

```
↓
```

OCR

↓

text

Repeat for every page.

Do NOT care which upload field it came from.

Step 3

LLM Page Relevance

This replaces the old classifier.

Instead of manually checking keywords,

the OCR text is sent to an LLM.

Prompt

You are reading OCR text from scholarship documents.

Determine whether this page is

1.

Bonafide Certificate

2.

Student Identity Card

3.

Neither

If neither

reply

NOT_RELEVANT

If relevant

extract

```
Official College Name  
Official College Address  
Return JSON only.
```

This single prompt replaces both
classification
and
structured extraction.

Why LLM Instead of Rule-Based Classification

Rule-based keyword scoring becomes brittle.

Examples

```
Bonafide Certificate  
Bonafide  
Certificate of Student  
Certified that  
Academic Year
```

All different wording.

LLM naturally handles variations.

It also ignores

Aadhaar

Ration

Declarations

without custom logic.

Desired LLM Output

For relevant pages

```
{  
  "relevant": true,  
  "document_type": "bonafide",  
  "college_name": "XYZ Institute of Technology",  
  "college_address": "Civil Lines, Nagpur, Maharashtra"  
}
```

For irrelevant pages

```
{  
  "relevant": false  
}
```

Nothing else.

Processing Flow

For every claim

Download files

↓

Normalize

↓

Every page

↓

OCR

↓

LLM

↓

Relevant?

↓

Collect extraction

Selecting the Best Result

A claim may contain

Bonafide

Bonafide duplicate

Student ID

Combined PDF

Multiple relevant pages may exist.

Selection strategy

Priority

Bonafide

↓

Student ID

If Bonafide extraction succeeds

ignore Student ID.

Only use Student ID if Bonafide cannot provide usable information.

Excel Update

Current columns

```
corrected_college_name  
corrected_college_address
```

Need

```
df.loc[index,  
"corrected_college_name"]  
  
=  
  
college_name  
  
df.loc[index,  
"corrected_college_address"]  
  
=  
  
college_address
```

Save periodically.

Recommended

```
Every 20 records
```

instead of once at the end.

Manual Review Flag

If

- OCR fails

- LLM cannot determine relevance
- no relevant pages found
- multiple conflicting college names

then

add

manual_review

YES

(or similar flag column)

instead of guessing.

Current Completed Modules

verify_colleges.py

Status

Complete.

Responsibilities

- Portal automation
- URL extraction
- Download documents
- Normalize documents

No further Playwright work is expected.

document_processor.py

Status

Mostly complete.

Responsibilities

- PDF rendering

- Image conversion
- Duplicate detection
- Page generation

Remaining

- automatic orientation normalization
-

classifier.py

Current status

Prototype only.

Expected to be removed.

Replaced by LLM page relevance detection.

Files Still To Build

ocr_engine.py

extractor.py

ocr_engine.py

Only wraps PaddleOCR.

extractor.py

Contains

- LLM prompt
- JSON parsing
- college extraction
- relevance decision

No browser logic.

Final End-to-End Pipeline

Excel

↓

Playwright

↓

Claim Form

↓

Extract URLs

↓

Download Uploaded Files

↓

SHA256 Duplicate Detection

↓

Normalize Documents

↓

Split PDFs into PNG pages

↓

Orientation Correction

↓

OCR Every Page

↓

LLM

↓

Relevant?

↓

Extract College Name

↓

Extract College Address

↓

Update Excel

↓

Periodic Save

Guiding Design Principles

1. Never trust the upload field.
2. Treat every uploaded document as a container of pages.
3. OCR every page.
4. Let the LLM decide whether a page is relevant.
5. Never use rule-based extraction for college details.
6. Skip duplicate files using SHA256.
7. Never stop processing because a document is missing.
8. Save Excel progress periodically.
9. Flag uncertain cases for manual review instead of guessing.
10. Keep Playwright completely isolated from OCR and AI logic.